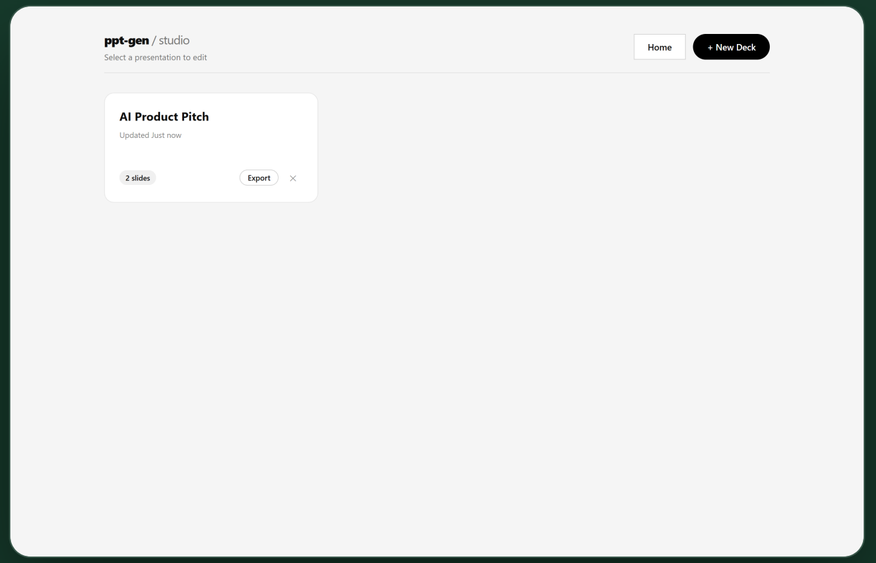
PROJECT REPORT

PPT-GEN

AI-powered presentation generator built with React.js, componentized layout schemas, and client-side multi-format export engine.



Presented by: **Vikrant Tewatia** (24001015068) & **Sushant** (24001015062)
Under the Guidance of: **Dr. Meenakshi** | Department of Electronics, J.C. Bose UST

Problem and Objective

The problem

Manual formatting is slow

Non-designer professionals spend up to 60–70% of total authoring time wrestling with coordinate placement, margin alignment, and visual inconsistencies rather than refining their core ideas.

Unstructured blank-canvas dilemma

Users are confronted with empty 16:9 canvases, creating cognitive friction and writer's block when attempting to formulate slide outlines from scratch.

Degradation of visual aesthetics

Free-form drag-and-drop mechanics allow arbitrary coordinate placement, resulting in broken hierarchies, mismatched margins, overlapping boxes, and visually unappealing slide decks.

Our objective

Prompt-to-slide schema synthesis

Decouple semantic content authoring from visual structural rendering, mapping prompts into strictly validated JSON data contracts.

Constrained responsive layout engines

Enforce mathematically proportioned 16:9 layout templates to preserve typographic hierarchy and prevent layout degradation.

Client-side multi-format export pipeline

Enable instant, high-fidelity exports into PDF, Word (.docx), PowerPoint (.pptx), images (JPG/PNG), and standalone web decks directly in the browser.

Technology Stack

Layer	Technology	What it does
Frontend Framework	React 19 + Vite	Component-driven single-page architecture with centralized state management and sub-second HMR.
Routing & Views	React Router v7	Multi-page declarative client-side routing across Landing (/), Dashboard (/projects), and Editor (/editor/:id).
Canvas & Styling	Unified CSS3	High-contrast monochrome palette, responsive grids, and locked 16:9 presentation canvas geometry.
PDF Engine	jsPDF + html2canvas	Off-screen 2x High-DPI canvas rasterization compiled into landscape multi-page PDF documents.
Document Export	docx (Packer)	Pure client-side Office Open XML builder generating structured Microsoft Word (.docx) documents with headers and page breaks.
Slide Deck Export	pptxgenjs	Direct client-side generation of native 16:9 widescreen Microsoft PowerPoint (.pptx) presentations.
Archive Compression	JSZip	In-memory client-side compression for bulk exporting all presentation slides as high-resolution image ZIPs.
Schema Contract	JSON Schema	Strict presentation schema models ensuring zero layout drift and reliable multi-format translation.

System Architecture

React Client
Landing, Projects Dashboard, and 16:9 Editor Views powered by React Router v7.

→

State & Canvas Engine
Unidirectional state store holding presentation arrays, active slide IDs, and markdown updates.

→

Virtual Staging Engine
Isolated 1280x720 off-screen DOM renderer ensuring artifact-free capture.

→

Multi-Format Exporters
jsPDF, docx, pptxgenjs, JSZip, and native Blob builders producing downloads.

Secure & Zero-Server Client Architecture

All document and image compilation executes 100% locally in the client browser, eliminating server storage costs, upload delays, and third-party data exposure.

Strict Schema Predictability

Layout components strictly adhere to predetermined responsive ratios, completely eliminating the overlapping boxes and margin bugs found in traditional slide suites.

How Slide Generation & Export Works

1

Topic Input

User enters a presentation topic or opens an existing deck with custom requirements.

2

Schema Map

Content structures into validated JSON objects specifying slide titles, content bodies, and timestamps.

3

Live Canvas

React renders responsive 16:9 slides with real-time text editing, thumbnail navigation, and deck renaming.

4

Virtual Stage

During export, each slide clones into an off-screen 1280x720 node to strip input borders and cursors.

5

Multi-Export

The chosen engine compiles clean slides into PDF, Word (.docx), PowerPoint (.pptx), images, or web decks.

By decoupling semantic content authoring from visual structural rendering, PPT-GEN guarantees 100% design consistency across all presentation outputs.

Data and Slide Schema Design

Presentation Deck Schema

- **id: string (e.g., 'proj-1', 'proj-172855...')**
  Unique identifier generated at creation, used for URL routing (/editor/:id).

- **title: string (e.g., 'AI Product Pitch')**
  Deck title editable directly from the editor navigation bar or dashboard cards.

- **updatedAt: string (e.g., 'Just now')**
  Timestamp showing last modification for dashboard sorting and project audit.

- **slides: Array<Slide>**
  Central array containing all sequential slide entities belonging to the deck.

Slide Entity Schema

- **id: number (e.g., Date.now())**
  Unique timestamp identifier ensuring reliable React key reconciliation and deletion.

- **title: string**
  Slide heading text, mapped to H1 headers in PDF, Word, and PowerPoint exports.

- **content: string**
  Multi-line body content, bullet points, and summaries preserved with pre-wrap whitespace.

- **layout: '16:9' aspect-locked container**
  Enforces mathematical proportions, preventing visual degradation across all displays.

Strict JSON data models eliminate arbitrary pixel coordinates, guaranteeing deterministic, bug-free document compilation.

Dashboard Overview

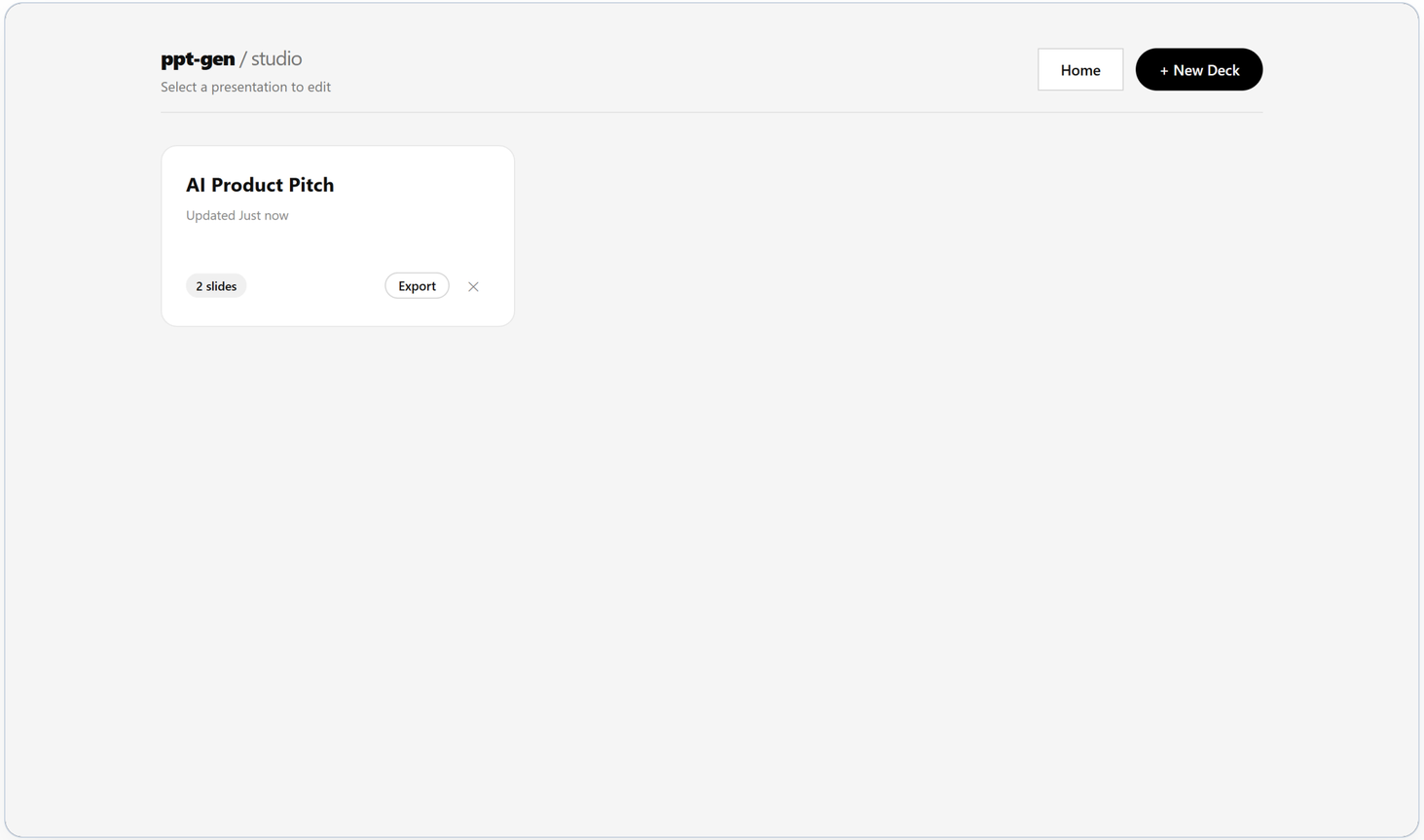


Figure 1: Projects dashboard screen (/projects)

What it shows

- Minimalist, high-contrast grid displaying all user presentation decks.
- Real-time metadata displaying slide count pill and last modified status.
- **+ New Deck** action button instantly generating fresh presentation workspaces.
- Quick **Export** button on each card for downloading decks without entering the editor.
- Safe deletion control (✕) allowing removal of outdated decks.

16:9 Presentation Editor

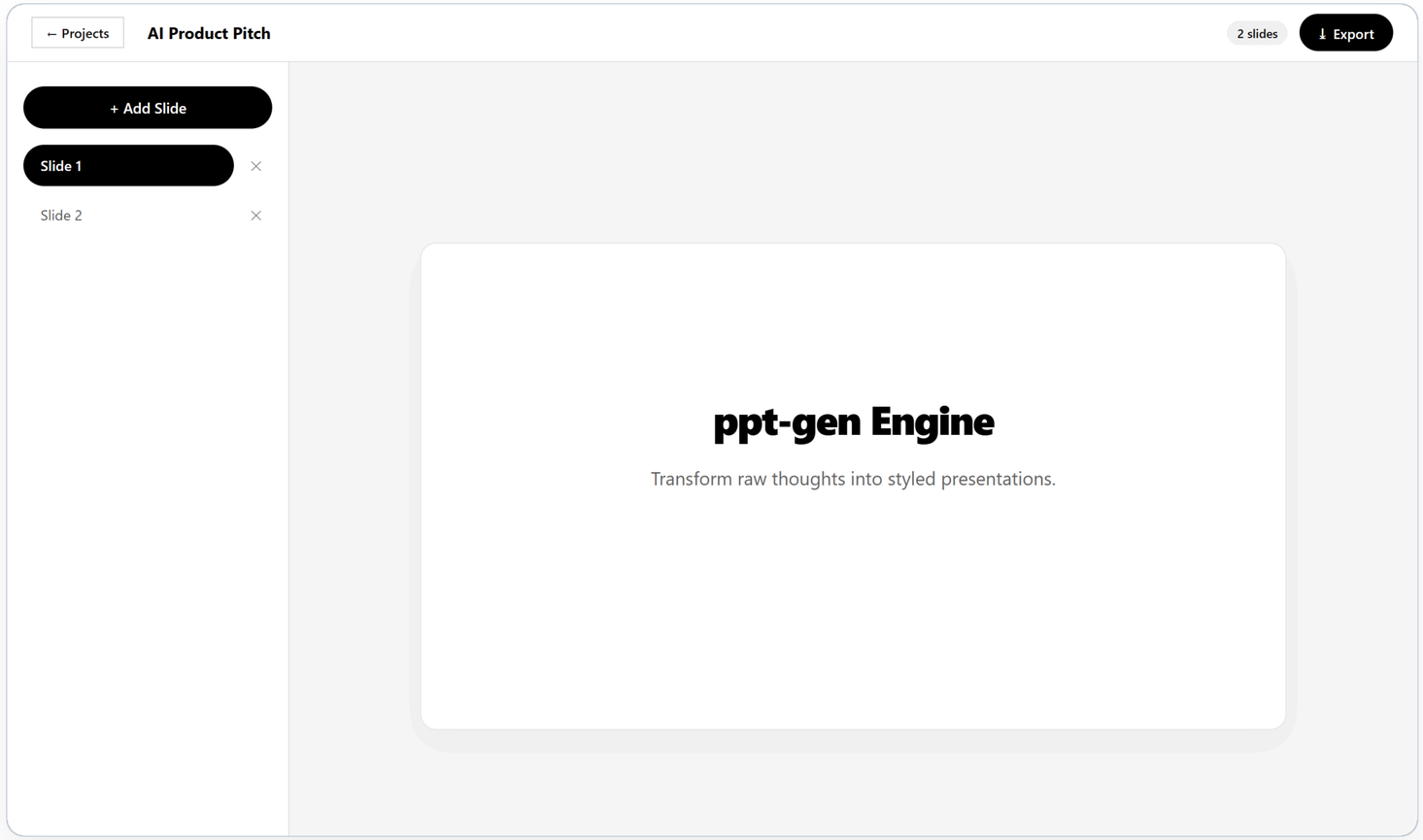


Figure 2: Slide editor workspace (/editor/:id)

What it shows

- Aspect-locked 16:9 presentation canvas preventing design stretching.
- Distraction-free inline editing for slide headlines and descriptive body text.
- Left thumbnail sidebar providing instant slide switching and overview.
- **+ Add Slide** control dynamically expanding deck size with instant state sync.
- Top navigation header featuring inline deck title renaming and export trigger.

Slide Navigation & Management

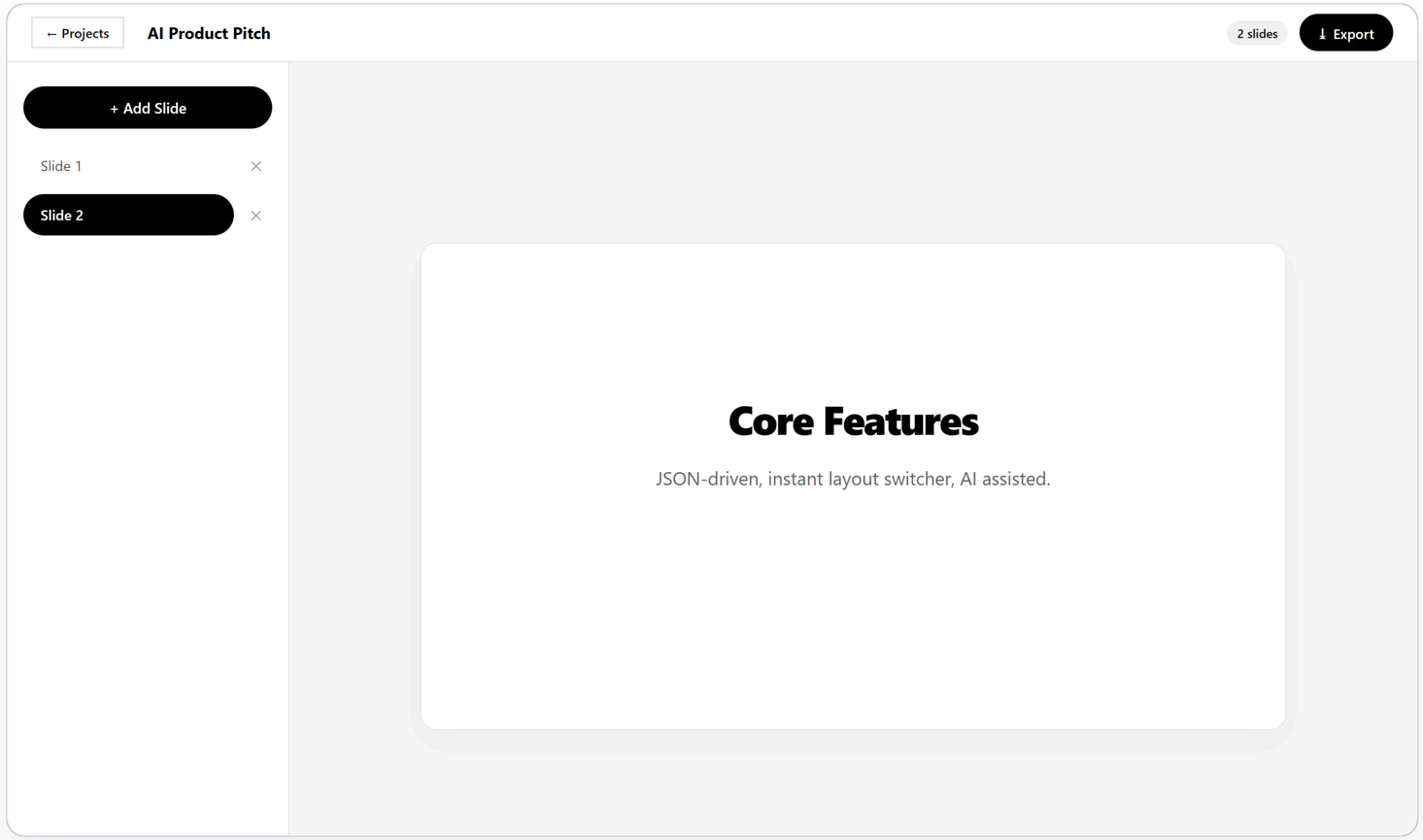


Figure 3: Multi-slide authoring view with active selection

What it shows

- Active slide indicator highlighting the currently edited slide in the sidebar.
- Smooth state switching between slides with zero re-rendering lag.
- Per-slide deletion button (✕) equipped with minimum-slide protection.
- Two-way text synchronization updating presentation state in real time.
- Subtle dashed input outlines appearing only during active focus or hover.

Multi-Format Export Engine

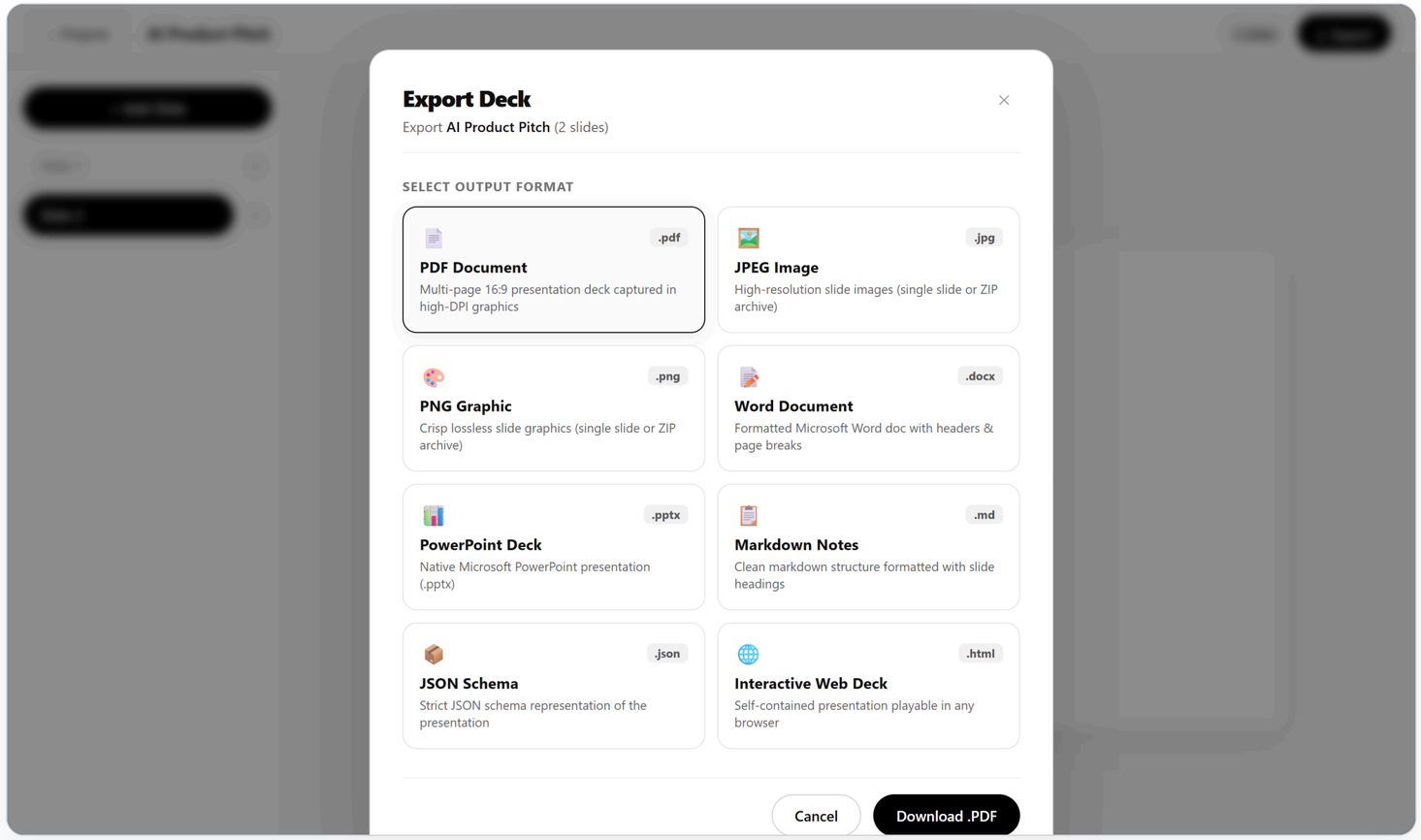


Figure 4: Multi-format export dialog with 8 output formats

What it shows

- High-contrast modal interface supporting **8 distinct file formats**.
- High-DPI multi-page PDF compilation via jsPDF and html2canvas.
- Image scope selector: Current Slide only or All Slides as a compressed ZIP.
- Native Microsoft Word (.docx) and PowerPoint (.pptx) file generation.
- Animated progress indicator with real-time feedback during canvas rendering.

Landing Page & Product Synopsis

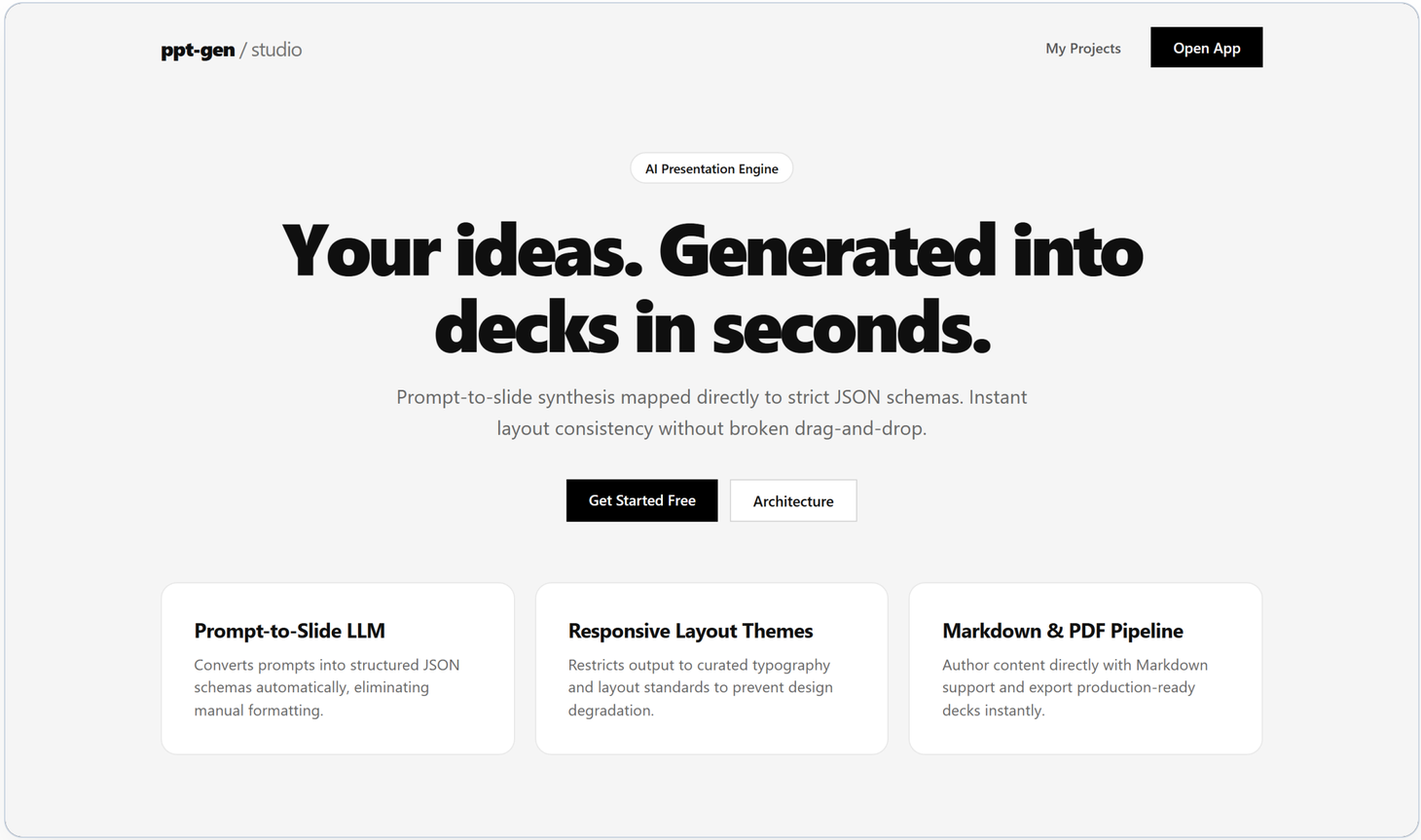


Figure 5: Landing page and architecture synopsis (/)

What it shows

- High-impact hero section presenting the AI presentation engine value proposition.
- Architectural callout cards highlighting Prompt-to-Slide LLM synthesis.
- Responsive layout standards card preventing design degradation.
- Clear call-to-action buttons directing users into the deck dashboard.
- Unified aesthetic matching modern minimalist design principles.

Challenges and Solutions

What went wrong	How we fixed it
Free-form drag-and-drop caused overlapping boxes & broken margins	Enforced mathematically proportioned, responsive 16:9 layout containers with CSS aspect-ratio locking.
Capturing live editor screen produced blurry text and input cursor artifacts	Architected an off-screen virtual stage rendering clean semantic HTML at 2x High-DPI resolution.
Only active slide was mounted in DOM, preventing multi-slide PDF export	Implemented sequential off-screen DOM staging to capture every slide sequentially into jsPDF and JSZip.
Client-side Word & PowerPoint export required heavy server roundtrips	Integrated client-side docx and pptxgenjs compilers, allowing 100% in-browser binary generation.
Nested React Router collision crashed navigation on page reload	Refactored App.jsx and main.jsx into a clean single BrowserRouter under StrictMode.
Monolithic CSS grew past 480 lines with maintainability bottlenecks	Restructured into 236 lines of clean, categorized rules without losing any styling or animations.

Conclusion and Future Scope

Conclusion

PPT-GEN successfully demonstrates how decoupling semantic content authoring from visual design solves the "blank-canvas dilemma". By combining React 19, strict JSON schemas, and a client-side multi-format export pipeline (PDF, DOCX, PPTX, Images, HTML), users can author, customize, and export presentation-ready decks in seconds without design degradation.

Future scope

- **Real-time LLM inference streaming API integration**

  Direct connectivity with Gemini / Claude APIs for automated prompt-to-deck streaming.

- **Automated AI image and chart generation**

  Contextual visual synthesis using text-to-image models and dynamic SVG data charts.

- **Collaborative multi-user editing**

  Real-time team co-authoring using WebSockets and CRDT synchronization.

- **Cloud presentations sync**

  Direct export integration with Google Slides API and Microsoft 365 cloud services.

Thank you. Questions?